



I404 - Reinforcement Learning

**Final Project:
Sequential KV-cache eviction with PPO**

Alexander Bodner
Ana Paula Tissera

5 July 2026

Artificial Intelligence Engineering

Contents

1	Introduction	1
2	Background	3
2.1	Learning to Evict from Key-Value Cache (Apple, KVP)	3
2.2	Neural Garbage Collection (Stanford, NGC)	4
2.3	<i>Eviction</i> heuristics as baselines	4
3	Proposed Directions	5
3.1	Sequential environment	5
3.2	Reward design	5
3.3	Experiment design	6
4	Development and Results	7
4.1	Metric and baselines	7
4.2	Starting point: reimplementing KVP	7
4.3	Building the environment and first experiments	8
4.4	The <i>chat template</i> problem	9
4.5	Why the policy did not beat kv_norm	9
4.6	A first capacity screen	9
4.7	Scaling the system: features, warm-start and attention	10
4.8	Wide eval and regime analysis	11
4.9	A policy with future information	12
4.10	Future attention is unpredictable from the present	13
4.11	Long generations and the exploration limit	13
4.12	The causal dense reward at scale	13
4.13	Per-layer credit	14
4.14	Per-layer reward and a second bottleneck	14
4.15	The dataset as the limiting factor	15
4.16	Our method on a dataset with signal	16
5	Conclusion	17
5.1	Contributions	18
5.2	Limitations and future work	18
6	References	18

Abstract

The KV cache of a transformer grows linearly with context length, and becomes the main memory bottleneck when the model reasons through long chains of thought. An alternative to fixed *eviction* heuristics (recency, attention sinks, key norm) is to *learn* which tokens to discard. Starting from Apple’s approach (KVP), which keeps the language model frozen and trains an external policy with a bespoke trainer, this work proposes reformulating *eviction* as a sequence of discrete decisions, one per decoding step, so that the problem becomes expressible as a Gymnasium environment and trainable with standard masked PPO. On GSM8K, the learned policy systematically lands at *parity* with the key-norm heuristic (**kv_norm**), despite exploring richer representations, different architectures, *warm-start* and corrections to credit assignment. Measured against a future-attention oracle, we show that this null result is a property of the dataset and not of the method. On that other dataset, our online formulation shows preliminary evidence of being able to beat **kv_norm**, something that never happened on GSM8K.

Keywords: *KV cache, reinforcement learning, eviction, PPO, transformers.*

1 | Introduction

Transformer-based language models generate text *autoregressively*. To produce the token at position t , the model conditions on the whole sequence of tokens already generated. In each layer, this conditioning is implemented through the *attention* mechanism [13], where every token computes, from its own internal representation, three vectors called *query*, *key* and *value*. To decide how much to “look at” each previous token, the *query* of the current token is compared against the *key* of every previous token through an inner product; those scores are normalised with a *softmax* into a distribution of weights summing to 1, and the layer’s output is a weighted average of the *values* of all previous tokens, using those weights. This operation is repeated in parallel across several attention *heads* per layer, each with its own projection of Q , K and V .

The *key* and *value* vectors of an already-processed token do not change: they depend only on that token and the ones before it, never on future tokens. Recomputing them at every new step would be redundant, so the standard practice is to compute them once and store them in a cache (**KV cache**), to be reused in subsequent steps. This way, generating token $t + 1$ only requires computing the *query*, *key* and *value* of the new token, and comparing it against the stored *keys* of all the previous ones, instead of recomputing the full sequence.

The cost of that cache, however, grows linearly with sequence length, so the longer the context or the reasoning chain, the more memory is required. In models that reason through *chain-of-thought*, where the answer may depend on a great many intermediate tokens, the KV cache ends up being one of the main limitations to scaling context [6].

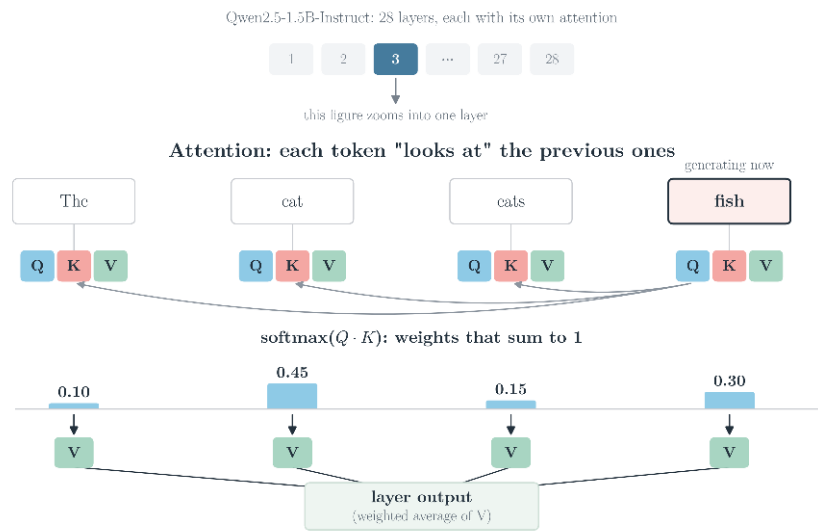


Figure 1.1: The attention mechanism inside a transformer layer. The *query* of the token being generated is compared against the *key* of every previous token; *softmax* turns those scores into weights summing to 1, and the layer’s output is a weighted average of the *values*.

The classical way to contain that growth is *eviction*, that is, discarding the least useful tokens from the cache in order to keep memory within a reduced budget. There are different approaches to deciding which tokens to keep. The most widespread techniques use fixed rules, such as keeping the most recent tokens and the first *attention sink*¹ tokens [14], prioritising *heavy hitters*² by accumulated attention weight [15], or selecting by the attention of a recent window [7]. While these strategies are cheap, they hand-code what is supposed to be important, instead of learning it from the task.

The alternative is to *learn* the *eviction* policy. Two recent works explore this path from different angles. Apple, in *Learning to Evict from Key-Value Cache* (KVP) [8], keeps the language model frozen and trains lightweight RL agents (one per attention head) that learn to rank cache entries by their future utility. Stanford, on the other hand, in *Neural Garbage Collection* (NGC) [6], fixes the form of the policy (it reuses the model’s own attention mechanism as the *scoring* function) and trains the language model to learn to reason and to forget simultaneously, with a single correctness reward signal. Both approaches are described in more detail in section 2.

Our starting point is Apple’s approach: keeping the language model frozen and training an external, lightweight policy to decide which entries to remove from the KV cache of an already-trained model.

The question motivating this work is whether that idea can be reformulated so that it can be trained with PPO [12], over a Gymnasium environment [2] and using Stable-Baselines3 [11], instead of depending on a specific implementation like Apple’s.

The difficulty motivating the reformulation is that in KVP the action does not fit that framework. Ranking or selecting a subset of the cache is a distribution over permutations (or over subsets) of variable size, and that is neither a discrete action (**Discrete**) nor a continuous one (**Box**). It therefore cannot be described as a Gymnasium space and, consequently, Stable-Baselines3’s PPO, which assumes a simple parametric distribution (categorical, Gaussian or Bernoulli) over a fixed-shape space, cannot train it. This is why Apple resorts to its own trainer (RLOO with the Gumbel-top-*k* trick³).

The central idea of this work is to **reformulate *eviction* as a sequential decision so as to adapt it to PPO**. Instead of ranking the whole cache in one pass, the agent discards *one* token per decoding step.

¹The first tokens of the sequence (usually between 1 and 4) receive, in almost every head and layer, a disproportionate amount of attention mass regardless of their semantic content: since the softmax must always sum to 1, the model uses them as a place to “dump” leftover attention when no other token is relevant. Evicting them from the cache degrades performance abruptly.

²The tokens that accumulate, over the whole generation (not just the current step), the largest summed attention weight. The heuristic identifies them and keeps them together with a window of the most recent tokens.

³The Gumbel-top-*k* trick makes it possible to sample, differentially, an ordered subset of size *k* from importance scores: independent Gumbel noise is added to them and the top *k* are taken, which also gives the exact log-probability of that subset in closed form. RLOO (*REINFORCE Leave-One-Out*) is the policy gradient estimator that uses that sample to train. Both are described in more detail in subsection 2.1.

The action at each step is a single **Discrete** choice over cache positions, but not all of them are evictable at all times (attention sink, recent window) and the set of occupied positions changes as tokens are evicted and generated. To solve this we used *action masking*⁴. This was implemented with **MaskablePPO**, a PPO variant included in the **sb3-contrib** library.

This reformulation has two main advantages. First, the problem becomes expressible in the standard reinforcement learning framework and can be trained using an existing PPO implementation. Second, instead of taking a single decision over the whole episode, the agent takes a decision at each generation step. This means obtaining a gradient signal at every step, which is especially useful when the reward is based solely on the final correctness of the answer.

Concretely, a policy was trained over Qwen2.5-1.5B-Instruct [10], frozen, to solve GSM8K problems [3] under a fixed cache budget. *Eviction* is performed *online* while the model generates the answer. In other words, the episode coincides with the generation itself and the agent must preserve the reasoning tokens that will be needed later, not only those that look important in the static prompt.

A Gymnasium-compatible environment was developed in which a single shared policy decides, independently for each layer, which token to discard at each decoding step. Finally, the performance of the learned policy was evaluated on GSM8K and compared against several reference heuristics (key norm, recency with attention sinks, random selection), as well as the ideal case in which there is no memory constraint.

2 | Background

This work takes as its reference two recent proposals that address the problem of learned KV-cache *eviction*. Both seek to solve the same problem: deciding which tokens to keep when the memory available for the cache is limited. Apple’s work keeps the language model frozen and learns an external policy to decide *eviction* (subsection 2.1), while Stanford’s keeps the policy fixed and trains the model itself to learn simultaneously to reason and to discard information (subsection 2.2). Our work builds on the former; the latter is the counterpoint that delimits the design space. Finally, we describe the main *eviction* heuristics, used as baselines in the experimental evaluation (subsection 2.3).

2.1 | Learning to Evict from Key-Value Cache (Apple, KVP)

Moschella *et al.* [8] propose KVP (*KV Policy*), a framework of lightweight RL agents that learn to rank KV-cache entries by their future utility, without modifying the language model or adding inference cost. Instead of retraining the model, it stays frozen and only a policy that decides which entries to keep in the KV cache is learned. Since this approach is the starting point of our work, in this section we describe the most relevant aspects of its formulation.

One characteristic of KVP is that it does not use a single policy for the whole cache. Instead, it trains *one agent per (layer, KV head) pair*. For Qwen2.5-7B-Instruct, with 28 layers and 4 KV heads, that is 112 agents; for the smaller 1.5B model, with 2 KV heads per layer, it is 56. Each agent is a small MLP that receives only the key and value vectors (K , V) of the cached positions (it uses neither the *queries* nor the hidden states) and produces an importance score per position. The agents are trained *offline*. First the model is run and the $Q/K/V$ activations per layer and head are saved to disk; then the policy is trained using only that data, without loading the language model again.

The reward used by KVP does not depend on whether the final answer is correct. Instead, it measures how well the policy manages to identify the tokens that will receive attention in future steps (*future attention*). To do this, it compares the ranking produced by the agent with the attention observed afterwards and computes a score similar to the area under the curve (AUC), evaluated across different cache budgets. The policy therefore receives a dense training signal instead of a single reward at the end of the episode.

The key point for this work is how the environment is formulated. In KVP, the *eviction* decision is taken **all at once**. Given the set of cached positions, the agent decides in a single pass which subset of size K to keep. From a probabilistic standpoint, this amounts to defining a distribution over the

⁴At each step, before sampling the action, zero probability is assigned to invalid positions (protected or non-existent at that moment) and the categorical distribution is renormalised over those that remain. The policy can therefore never choose a forbidden position, without needing to change the size of the action space.

possible rankings of the positions, modelled with a Plackett-Luce distribution [9] over permutations. In practice, this corresponds to sampling an ordered subset of size K without replacement. To make it differentiable and admit policy gradients, KVP samples that subset with the *Gumbel-top-k* trick [4], which yields the exact log-probability of the ordered subset in closed form, and trains with RLOO (*REINFORCE Leave-One-Out*) [1]⁵.

This formulation means the problem cannot be represented directly with the action spaces available in Gymnasium and, therefore, cannot be trained with Stable-Baselines3’s PPO implementation. First, the action consists neither of choosing a single position nor of producing a continuous vector, but of selecting an ordered subset of positions. Moreover, the decisions are not independent of one another, since the probability assigned to a position depends on which ones were chosen before. Finally, the number of available positions changes during generation, whereas Gymnasium requires fixed-size action spaces.

Our goal is to bring KVP’s idea into a Gymnasium-compatible environment that can be trained using PPO. The proposed reformulation is described in section 3.

2.2 | Neural Garbage Collection (Stanford, NGC)

Li *et al.* [6] approach the problem from a different angle and, for that reason, it serves to delimit the central design decision of this work. Instead of freezing the language model and training an external policy, their *Neural Garbage Collection* (NGC) method **keeps the form of the policy fixed and trains the language model itself**. They teach the model to “forget while it learns to reason”: as it generates its chain of thought, the model periodically enters an *eviction* round, scores the cache entries, discards part of them and continues reasoning conditioned only on what remains.

This contrast is what justifies our choice of starting from KVP and not from NGC. Training the model end-to-end as NGC does requires backpropagating through the whole language model at every step, which demands that the model be trainable and carries a much higher compute cost. KVP’s route, by contrast, keeps the model *frozen* and as a black-box oracle, and only trains a small policy: it is cheaper and, above all, applicable to any already-trained model without touching its weights. Our work follows that decision, and this is why the model stays frozen in every experiment.

2.3 | Eviction heuristics as baselines

Both KVP and NGC compare their learned policies against fixed heuristics, that is, rules that decide which tokens to keep without any training. In this work we use the same heuristics as a reference to evaluate whether a learned policy really offers an improvement.

One of them is *StreamingLLM* [14], which keeps the first tokens of the sequence (the *attention sinks*, towards which the model tends to direct attention systematically) together with a window of the most recent tokens, and discards everything else.

The main baseline used in this work, and the one we aim to beat in the experiments (section 4), is the *key norm* heuristic (*kv-norm*). At each generation step, this heuristic discards the cache entry whose value of $\|K\| + \|V\|$ is smallest, that is, the one with the lowest combined magnitude between its key vector and its value vector. It is a simple heuristic, it requires no attention signal, and it turned out to be very hard to beat.

H2O [15] identifies the *heavy hitters*, which are the tokens that accumulate, over the *whole* generation, the largest summed attention weight, and discards the rest. This heuristic was implemented as an accumulated-attention oracle and measured on GSM8K, but it was found not to beat *kv-norm* in a statistically significant way (subsection 4.9). The reason is that attention over the question does not anticipate what needs to be kept from the reasoning that has not been generated yet. For this reason it is not considered the heuristic to beat in this work.

SnapKV [7] assigns scores to positions using the attention of a recent window of *queries*. In this work we do not use SnapKV as a baseline, but we adopt an aggregation criterion from KVP for models with

⁵RLOO samples several trajectories for the same input and, for each one, uses as a *baseline* the average of the rewards of the *other* samples in the group (leaving its own out). This reduces the variance of the REINFORCE gradient without having to train a separate critic network.

grouped-query attention (GQA)⁶: instead of averaging, KVP takes the maximum over the *query* heads within each group, and we apply that same rule to our own importance *proxies*.

In summary, these heuristics require no training and are computationally cheap, but they embed a fixed notion of importance.

3 | Proposed Directions

This section describes the main ideas this work started from, building on Apple’s KVP repository. From the beginning, the approach was shaped by two goals. The first was to reformulate the *eviction* problem so that it could be represented as a sequential environment and trained with PPO. The second was to define an evaluation methodology based on paired comparisons and on experiments designed to answer concrete questions about the policy’s behaviour.

The third decision, concerning the reward used to train the policy, was not settled from the outset. KVP starts from a dense signal (future attention), and at an early stage we replaced it with a correctness reward so as to be able to use the fast attention backends, which introduced the problem of reward sparsity. Only later in the project did we arrive at a dense, causal signal that recovers that density without paying the cost of explicit attention.

section 4 presents the experiments that grew out of these design decisions.

3.1 | Sequential environment

As anticipated in section 1, the main idea of this work is to turn the *eviction* decision into a sequence of discrete actions, one per decoding step. This way, the problem can be represented as a Gymnasium environment and trained using Stable-Baselines3’s `MaskablePPO`.

The episode coincides with the generation of an answer. First the prompt is *prefilled*⁷ and, from that moment on, the model generates one token at a time. When the KV cache exceeds the fixed memory *budget*, the agent has to decide which token to remove before generating the next one. The cache size therefore stays constant throughout generation.

Unlike KVP, where the decision is taken once over the contents of a precomputed cache, in this environment *eviction* happens while the model is generating the answer. As a consequence, the policy has to learn to preserve the tokens that will still be useful later in the reasoning, and not only those that look important in the initial prompt.

The transformer has 28 layers and the environment exposes each (episode, layer) pair as a sub-environment. A *single* shared policy decides independently which token to discard in each layer, this being the finest granularity the standard KV-cache implementation allows.

The reward is assigned when the episode ends (because it relates to the correctness of the answer) and training uses Monte Carlo returns ($\gamma = 1$, $\lambda = 1$). Without discounting, each decoding step contributes equally to the outcome. Which signal to use as the reward is a decision described in the next section.

Compared with KVP, where the policy takes a single decision per episode, this reformulation yields a decision, and therefore a gradient signal, at every generation step.

3.2 | Reward design

With the environment defined, the next question was which reward to optimise. This decision kept changing over the course of the project as new experiments were run.

At first we followed KVP’s proposal and used a reward based on future attention (subsection 2.1). However, this approach forced us to use the `eager` attention backend, since explicit access to the attention weights was needed. That prevented us from taking advantage of optimised implementations such as `FlashAttention` or `sdpa`, considerably increasing the computational cost.

⁶In GQA, several *query* heads share a single key/value head, and therefore a single physical KV cache (see Figure 4.1 in subsection 4.3, which shows the concrete case of Qwen2.5-1.5B-Instruct). When computing an importance score per position, the *query* heads of the group must be combined into a single number.

⁷*Prefill* is the initial stage of inference in which the model processes all the tokens of the input prompt to build the KV cache. From that state, autoregressive generation begins, adding one new token at each step.

For this reason we replaced that reward with one based on the correctness of the final answer on GSM8K (1 or 0). This change allowed us to use the fast backend, but introduced problems of its own. This reward is extremely sparse, since it is only known at the end of the episode and, using Monte Carlo returns, the same return is assigned to every decision taken during generation (hundreds of steps and 28 layers). As a consequence, temporal credit assignment is very weak, since every *eviction* receives exactly the same training signal regardless of the effect it had on the generation.

To improve credit assignment we explored several alternatives, first incorporating penalties related to generation length and truncation.

With pure correctness, when the model never managed to emit the end-of-sequence token within the step budget, every episode in a batch truncated and correctness was 0 in all cases alike. In that regime, PPO had no gradient to learn from. To avoid this, a dense reward based on generation length and on whether the episode truncated was added:

$$\text{score} = \text{correctness} - w_\ell \cdot \frac{\text{steps}}{\text{max_new_tokens}} - w_t \cdot \mathbb{1}[\text{truncated without EOS}],$$

with $w_\ell = 0.5$ and $w_t = 1.0$ (set from knowing that the average GSM8K answer length with Qwen-1.5B was 500-700 tokens). Between two equally incorrect episodes, the shorter one is penalised less than the longer one, giving PPO a signal to shorten generation before it ever finds a correct answer. The truncation term penalises going round in circles without finishing, so that it is clearly worse than any *eviction* strategy that lets the model reach the end.

The final proposal of this work is a dense, causal reward that evaluates, at each step, the immediate effect of the *eviction* on the model’s predictions.

The idea is to compare, at each generation step, the next-token distribution obtained using the evicted cache with the distribution produced by a “shadow” cache that never evicts. The latter acts as a reference and makes it possible to measure how much each decision changes the model’s behaviour. The per-step reward is defined as

$$r_{\text{step}} = -w \cdot \text{clip}(\text{KL}(p_{\text{full}} \| p_{\text{evict}}), 0, c).$$

Each decision is thus rewarded or penalised according to the immediate effect it has on the probability distribution of the next token, without waiting for the episode’s final outcome and without needing an attention-based signal. Moreover, this reward is compatible with the fast `sdpa` backend, since it only requires the model’s output probabilities.

The motivation behind this proposal is that a denser training signal should ease temporal credit assignment and allow more effective *eviction* policies to be learned than with final correctness alone. In section 4 we evaluate this hypothesis experimentally.

3.3 | Experiment design

The second design decision concerned how to evaluate the learned policies. Instead of comparing only the accuracy obtained by each method, we use as our main metric the paired contrast between the learned policy and the `kv_norm` heuristic,

$$\text{correct}_{\text{learned}} - \text{correct}_{\text{kv_norm}},$$

computed over the same examples and with the same *anchors* (baselines) per run.

The main reference is `kv_norm`, the heuristic that removes the token with the lowest value of $\|K\| + \|V\|$. In addition, each experiment includes two further references: `full`, which uses the complete cache and represents the best possible performance under this configuration, and `random`, which removes tokens at random and acts as a lower reference.

The paired comparison reduces the effect of each example’s intrinsic difficulty and avoids comparing raw numbers across runs that may have evaluated on different subsets.

On this basis, the experiments were designed to answer concrete questions. Instead of modifying several aspects of the system at once, each experiment evaluates a specific hypothesis about the policy’s possible limitations, such as state representation, exploration, architecture or the reward used. section 4 follows that path and presents the experiments in the order they were carried out.

4 | Development and Results

This section presents the experiments carried out and the results obtained. The experiments are shown in the order in which they arose during the development of the work, since each one motivated the next.

4.1 | Metric and baselines

Every number that follows is on GSM8K with Qwen2.5-1.5B-Instruct and a cache budget of 256 tokens (unless stated otherwise). As anticipated in section 3, the main metric is the paired contrast $\text{correct}_{\text{learned}} - \text{correct}_{\text{kv_norm}}$ over a held-out *probe* of examples, with the fixed baselines **full** (complete cache, no eviction, the ceiling), **kv_norm** (best heuristic) and **random** (floor) evaluated on the same examples.

We say the learned policy reaches **parity** with **kv_norm** when the paired contrast is statistically indistinguishable from zero, that is, the policy gets *the same number* of problems right as the heuristic on the same examples, within measurement noise. Since the goal of the work was to *beat kv_norm*, parity is a negative result with respect to that goal, and much of this section consists of understanding why the policy systematically stayed there. One *probe* example is worth $1/128 \approx 0.008$ on this scale, so a gap of, say, -0.02 means “two or three problems below **kv_norm** out of 128”.

An important detail is that the raw numbers depend on the software *stack* (**torch/transformers**) and on the concrete subset of GSM8K, so *only the paired contrast within a single run is comparable*. Over the course of the project we saw up to five “regime shifts” where the same model and the same examples gave different absolute accuracies depending on the attention backend; this is why raw numbers are never compared across runs, only paired gaps within each one. If the policy-independent baselines (**kv_norm**, **random**) do not agree between two runs one wants to compare, that alone is enough to know they are not evaluating under the same conditions, and any raw difference between policies stops being interpretable.

4.2 | Starting point: reimplementing KVP

The work began by reproducing Apple’s method (KVP) on Qwen2-1.5B-Instruct. While Apple trains and evaluates on Qwen2.5-7B-Instruct, we used a smaller model from the previous generation so as to validate the implementation with reasonable training times on one GPU (they use 8 H100s). The goal of this stage was to verify that the reproduction of the method was correct, not to obtain results directly comparable with the original work.

To that end, we trained the offline rankers per (layer, head) pair with RLOO and the Gumbel-top- k trick, and evaluated across different cache sizes. This first reproduction was done, as in KVP, on **RULER**: a synthetic long-context benchmark where the prompt contains a specific fact hidden (a “needle”) in the middle of a lot of irrelevant text, and the task is to retrieve that fact at the end. RULER is a good test bench for KVP’s future-attention reward (subsection 2.1) because the right datum gives a clear answer about which prompt positions *should* receive attention (those of the needle) and which should not (the filler).

When reformulating the problem as a sequential task trainable with PPO we also changed datasets. Instead of RULER we used **GSM8K**, a set of maths problems that require a step-by-step reasoning chain to reach the answer. The change was motivated by the reward we wanted to optimise. In KVP the training signal evaluates how well the ranking predicts future attention, a dense but indirect reward, independent of whether the final answer is correct. We wanted to train directly against the correctness of the answer, for which we needed a task whose result could be verified automatically.

Moreover, the kind of information that has to be preserved also changes. In RULER the important information is already present in the prompt, whereas in GSM8K it appears during generation, as the model builds the reasoning chain. This made GSM8K look like a better choice for studying eviction policies during generation and not only over a fixed context. Later on (subsection 4.15) this very choice turned out to have a consequence we had not anticipated.

The next step was to develop the reformulation proposed in this work (section 1). Instead of ranking the whole cache at once, turn eviction into a *sequence* of decisions, one per decoding step, each one a **Discrete** choice with *action masking*. That reformulation is what makes the problem trainable with sb3-contrib’s **MaskablePPO**, and is the object of all the experiments that follow.

From this point on, every experiment was carried out using Qwen2.5-1.5B-Instruct. A newer architecture such as Qwen3.5 was not a viable option for this formulation, since they use a hybrid design in which only one layer in four has standard full attention, while the other three use *Gated DeltaNet*, a linear attention mechanism that computes neither *softmax* attention weights nor a token-indexed KV cache, but a fixed-size recurrent state [5]. Qwen2.5 uses standard full attention in all 28 layers, uniformly.

4.3 | Building the environment and first experiments

The next step was to implement the environment proposed in sections 1 and 3. Building it took several iterations before we had something trainable. Starting from a routine that runs the prompt's *prefill* over the frozen model and extracts the resulting KV cache (`capture.py`), the environment was built as a Gymnasium class compatible with Stable-Baselines3's `VecEnv`. At each step, the environment takes care of automatically resetting the episodes that have already finished (because the model generated an EOS token or hit the step limit) and of updating the valid-action mask used by `MaskablePPO`.

An architectural detail is that Qwen2.5-1.5B-Instruct uses *grouped-query attention* (GQA): twelve *query* heads but only two *key/value* heads per layer (six *query* heads share a single physical cache, figure 4.1). Since the KV cache exists only for the latter, the eviction decision has to be taken at the KV-head level, not the *query*-head level.

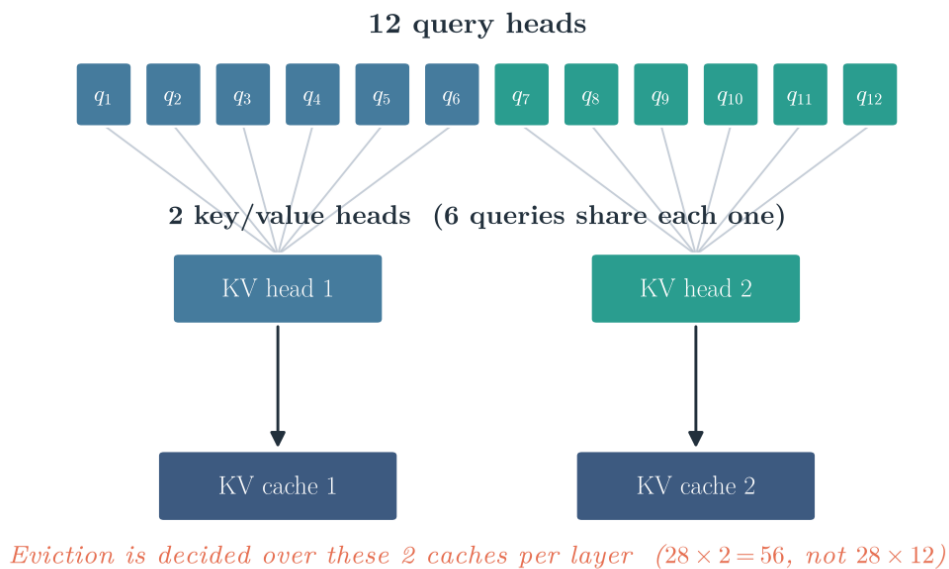


Figure 4.1: Grouped-query attention in Qwen2.5-1.5B-Instruct. The twelve *query* heads of a layer are grouped in sixes; each group shares a single *key/value* head, and therefore a single physical KV cache. Eviction can only be decided at the level of those two physical caches per layer, not per *query* head.

The first version of the environment considered one sub-environment per (layer, *query* head) combination, that is, $28 \times 12 = 336$ in total. However, since Qwen uses GQA, there are only $28 \times 2 = 56$ independent key/value caches on which eviction can actually be performed. Correcting this reduced the environment to those 56 sub-environments. Later, moving from one policy per head to one policy per layer simplified the number of decisions even further.

Another problem appeared when the model generated an EOS token before the cache exceeded the budget. In those cases no eviction ever took place, but Stable-Baselines3 still recorded a transition with an observation, an action and a reward. Since that action never had any effect on the model, it ended up adding noise to training. To avoid this, the environment detects these episodes and discards them before they reach PPO.

Eviction was implemented by directly modifying `transformers`'s `DynamicCache` (Hugging Face's standard structure for the KV cache) layer by layer, instead of using *hooks* on attention, and updating the `position_ids` to keep the positional encoding (RoPE) consistent after removing a token.

It was also necessary to define what information the policy would observe. The first version used only three features per token: $\|K\|$, $\|V\|$ and the relative position $t/\text{max_len}$. Later this representation was replaced with the full K and V vectors (the entire `head_dim`, not just their norm), and the explicit position feature was dropped because the RoPE rotation is already in K . This was done on the hypothesis that a richer representation would make it possible to learn strategies going beyond the `kv_norm` heuristic. As will be presented in subsection 4.5, this decision also introduced an unexpected effect. Normalising those vectors in the network's first layer destroyed precisely the magnitude information that heuristic uses, and it had to be reintroduced explicitly as a separate feature (`kvz`) only at the scaling phase.

With the environment built, the first family of experiments explored which reward to use. We initially tried three signals: `none` (terminal correctness only, 1 or 0), `recency` (a dense per-step reward that rewards keeping recent tokens) and a *per-step attention* one (rewarding the eviction of tokens with low accumulated attention).

The main result of this stage was identifying a failure mode common to all these variants. When the policy could remove any token from the cache, training quickly collapsed towards strategies that discarded essential information and the generation degraded into repetitive text. To avoid this behaviour, protection of the *attention sinks* and of a window of recent tokens was added, following the idea of StreamingLLM (subsection 2.3) and H2O. This change stabilised training and left a first version of the environment ready for the comparative experiments of the following sections.

4.4 | The *chat template* problem

Before being able to trust any number, a problem that invalidated all the initial runs had to be solved. The model is Qwen2.5-1.5B-**Instruct**, trained to receive its prompts in a specific conversation format known as a *chat template*. Passing it the GSM8K question as plain text, without that format, meant the model **never emitted the end-of-sequence token** (EOS): it kept generating indefinitely until hitting the `max_new_tokens` limit. As a consequence, *every* episode ended by truncation and the correctness-based reward was always zero, which left PPO with no gradient to learn from. It took a while to detect this problem. At first we attributed it to a policy “cold start” problem, so we tried different alternatives, such as training only with easy examples or protecting the entire prompt from eviction. Only afterwards did we see that none of those changes attacked the real cause.

4.5 | Why the policy did not beat `kv_norm`

With the environment built and the *chat template* problem solved, we trained a first policy at scale with the correctness reward and measured its performance with the paired metric of subsection 4.1. The result was that the policy stayed at levels similar to `kv_norm`, with no consistent improvement over the heuristic. Instead of continuing to tune hyperparameters, we reviewed the implementation of the environment and the policy to understand what was limiting learning, and several possible causes appeared.

The first is that the policy loses magnitude information when a `LayerNorm` is applied over K and V as the network's first step, which removes the scale that `kv_norm` uses ($\|K\| + \|V\|$). The second is that there is no explicit position representation, and one depends solely on the information implicit in RoPE inside K , which is not always enough for the policy's architecture (a network that processes each token separately after normalising). The third is that the decision is taken independently per token, which limits the ability to capture relations between positions (for example, redundancy between tokens). The fourth is that training starts entirely from scratch, without any initialisation close to a reasonable policy, which makes exploration inefficient.

The work plan (section 3) was to isolate each of these possible limitations and evaluate its impact separately. The following sections follow exactly that path.

4.6 | A first capacity screen

Before spending GPU-days on scaling, we ran a cheap *screen* of each variant over the same set of 16 examples, evaluating on *those same* examples. It is a *capacity* test to find out whether that variant *can* beat `kv_norm` on data it has seen. figure 4.2 summarises the best gap reached by each one.

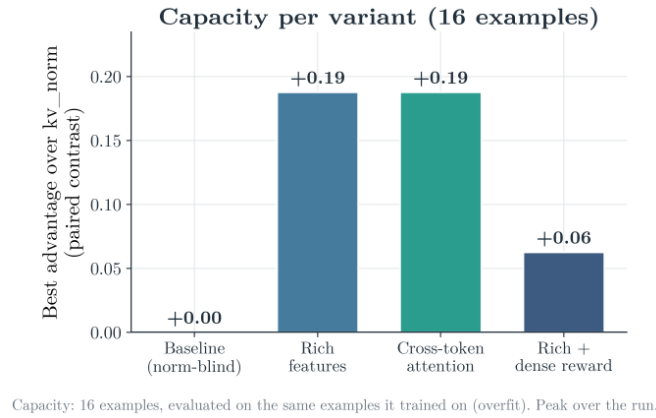


Figure 4.2: Capacity screen (16 examples, *overfit*). The paired contrast $\text{correct}_{\text{learned}} - \text{correct}_{\text{kv_norm}}$ is shown for different policy variants. The base version (without magnitude features) stays at levels similar to kv_norm . Adding richer features shows a marked improvement in peak performance. Adding the dense reward on top of an already expressive policy does not improve results in this configuration and tends to reduce performance.

Directionally, representation was still the main axis. We had two observations. The signal was fairly noisy and spiky, with jumps between 0 and 0.25. Moreover, adding the dense reward did not help even an already capable policy. With rich features + dense reward we obtained +0.06, below using rich features alone. At that point our contribution was starting to lose weight and become less clear.

4.7 | Scaling the system: features, warm-start and attention

The *screen* essentially told us what was worth scaling and what was not. From there we ran three arms at scale (2M steps, 1000 training examples and a held-out *probe* of 32), each one to isolate a different hypothesis. One variant with more informative features (to attack the absence of signal about norm and position), another with warm-start (cloning kv_norm via *behaviour cloning* before doing RL, to see whether the problem was the cold start), and a third with cross-token attention (to see whether introducing interaction between tokens helped capture relations that do not appear when each token is processed in isolation). Figure 4.3 shows the paired gap over the course of training.

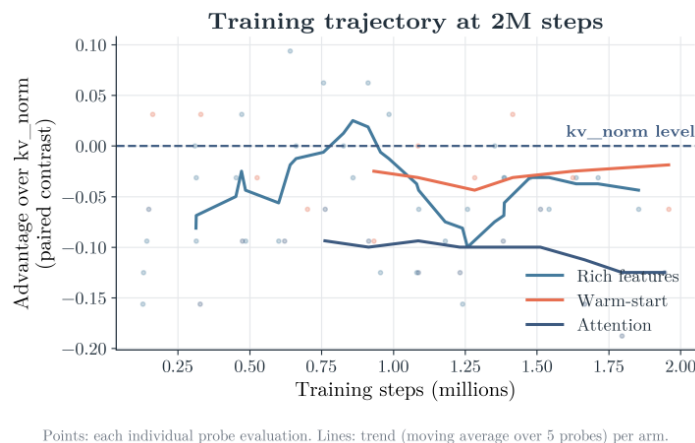


Figure 4.3: Runs at scale (2M steps). Paired contrast $\text{correct}_{\text{learned}} - \text{correct}_{\text{kv_norm}}$ vs. *timesteps*. The three arms oscillate around or below 0. The warm-start variant starts at kv_norm (the clone worked) but RL does not push it beyond; the attention variant stays systematically below.

The variant with more informative features closed at -0.044 on average (*last-5* -0.050), meaning they took the policy from $\approx$ random up to kv_norm (real progress on representation) but did not push it

beyond. When we include the exact `kv_norm` signal (`kvz`) as a feature, the easiest thing for the policy is to imitate it, and there it stalls.

The warm-start variant isolates the cold-start hypothesis. Instead of starting from scratch, we first clone `kv_norm` with behaviour cloning and then do PPO. If the problem were insufficient exploration, this should unblock it. The clone worked (the first *probe* has learned $\geq$ `kv_norm`), but even so adding 2M of PPO finds nothing better (it ends at -0.019).

The cross-token attention variant behaves worst, closing at -0.111 . The positive peak we had seen in the *screen* ($+0.19$ on an overfit of 16 examples) does not appear here. It seems, rather, to introduce complexity that PPO cannot exploit.

4.8 | Wide eval and regime analysis

Since the three arms lived on different *stacks*, the decisive number came from a wide evaluation: 128 new examples, taken from the stretch of the GSM8K shuffle after the training set and the 32-example *probe* used during training (that is, disjoint from everything any of the *checkpoints* saw), with the three final *checkpoints* evaluated on the *same* VM with the same *anchors* (figure 4.4).

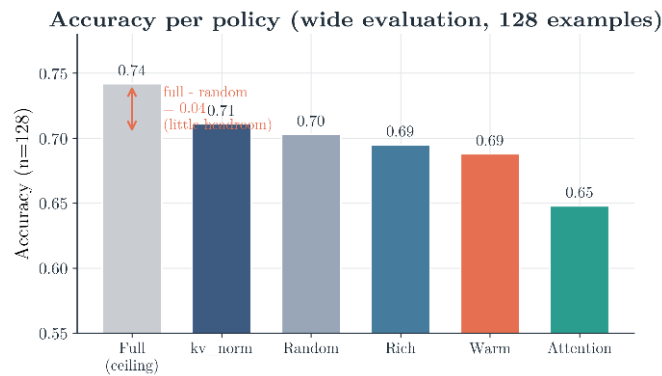


Figure 4.4: *Wide eval* ($n = 128$, a single *stack*). The per-token arms (rich features, warm-start) land 2 to 3 examples from `kv_norm` (parity within noise). The attention variant is consistently worse. The arrow marks the regime insight: $\text{full} - \text{random} = 0.039$.

This experiment turned out to be the most important discovery of the whole phase, and not because of the policies but because of the **regime**. That is, with this budget and these prompt lengths, evicting at random costs only $\sim 4\text{pp}$ relative to not evicting, and `kv_norm` captures barely $\sim 20\%$ of that already tiny margin. This puts a very low ceiling on any possible improvement. The terminal reward is nearly flat as a function of the policy, so PPO has no gradient to follow, and even a perfect oracle would win at most $\sim 3\text{pp}$.

This is, in part, an **architectural** limitation of the *training* environment. To train at a reasonable speed, the environment packs N episodes into a single *tensor* and does one forward pass of the model per step instead of N separate ones; for that to work, all the episodes of a batch have to share the same cache size, fixed at `budget+1`. Since eviction only acts on tokens generated during decoding (never on the prompt during *prefill*), an example is only trainable if its entire prompt fits within that budget; if it is longer, the environment simply discards it. In practice, this sets a floor on the budget usable during training, since it cannot go below the length of the prompts one wants to keep training on. This restriction belongs to the batched training environment; the single-episode-at-a-time evaluations we use later do not have it, and this is why there we can measure budgets below the prompt length. We call this unfiltered GSM8K *setup*, the one we had been using by default until now, the **low-difficulty regime**. figure 4.5 contrasts it with the long-generation one: the same GSM8K but filtered to the examples with the longest reasoning, where the learnable margin is larger.

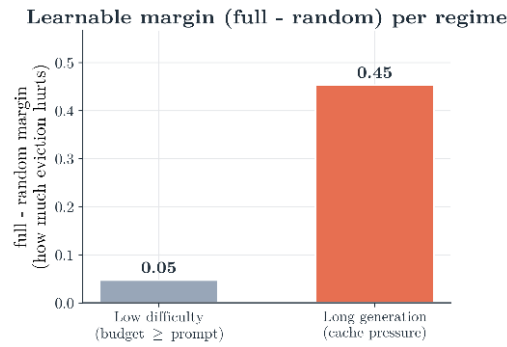
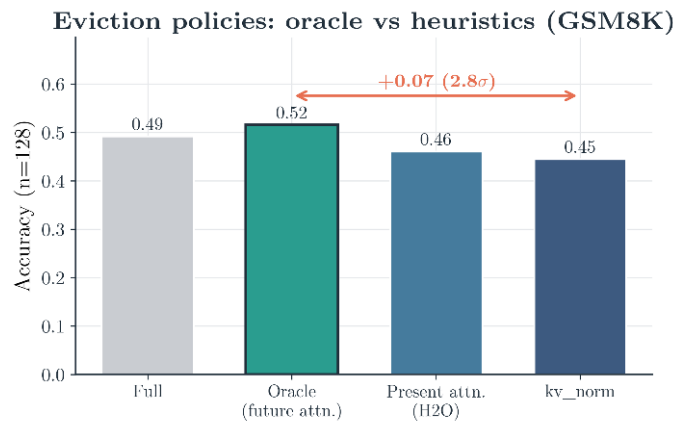


Figure 4.5: The learnable *headroom* depends on the regime (both on GSM8K). In the simple regime (budget $\geq$ prompt) full – random ≈ 0.04 ; in the filtered long-generation one, ≈ 0.45 , and there `kv_norm` does not even beat chance. The arena where eviction really matters is the second one.

4.9 | A policy with future information

If the low-difficulty regime has little *headroom*, the question is how much can be improved in principle. For that we built an ideal eviction with future information. Over the full generation trace, at each step the token with the lowest maximum future attention is discarded, a decision that is only possible with access to the future (figure 4.6).



GSM8K, eager backend. Present attention does not beat `kv_norm`; only the future-attention oracle does.

Figure 4.6: Eviction with future information ($n = 128$, *eager*) (oracle). The future-attention-based policy beats `kv_norm` by $+0.070$ (2.8σ) and even beats the full cache. Present-time attention (`attn_cur`) does not beat `kv_norm`. The margin exists but depends on future information.

There are three key points. First, the learnable margin exists ($\approx +7\text{pp}$) but it is future information. Present attention does not beat `kv_norm`. This explains the pattern that repeats through the rest of this phase, *on GSM8K in this regime*: since no online policy (learned or heuristic) has access to the future, they are all bounded by the same effective ceiling, which is `kv_norm`.

This policy with future information is not, however, a ceiling that real supervision can reach, but an upper bound computed with *hindsight information*, looking at the complete generation trace before deciding what to evict at each past step. Later we found that this was specific to this dataset. In the *passkey* setting (subsection 4.15), `kv_norm` stops being a ceiling and falls below even chance.

Second, the method with future information even beats the full cache ($0.516 > 0.492$), which suggests that a good eviction not only preserves information but can improve reasoning by discarding the right tokens.

Third, another regime shift appears. With *eager* attention the full cache scores 0.49 against 0.74 under *sdpa* (bf16), so the gap depends heavily on the attention regime.

4.10 | Future attention is unpredictable from the present

The natural next step, and the literature’s recipe, was to distil that oracle through behaviour cloning over the future-attention rankings, followed by a short RL fine-tune. The concrete question is whether a policy that only observes the present (K , V and derived features) can anticipate which token it is best to evict according to future information.

To measure this we used the `match_oracle` rate, defined as the fraction of steps in which the policy’s action coincides with the oracle’s. *Behaviour cloning* shows no convergence on this objective: `match_oracle` stays between 0.018 and 0.036 over the 3000 cloning steps. By contrast, the same recipe applied to `kv_norm` reached almost perfect agreement (≈ 1.0), which indicates that the cloning procedure is stable when the signal is accessible from the present. The +7pp margin is therefore made of information that is not contained in the observation of the present state, which makes the problem effectively one of partial observability with respect to that criterion. (In the literature they add attention-history features or other additional inputs that our `sdpa` environment does not expose, and even in those cases they require pairwise ranking schemes.)

4.11 | Long generations and the exploration limit

The *wide eval* had already shown where the *headroom* was: it actually appears only in long generations under cache pressure (figure 4.5). We moved the environment to that setting (`min_answer_words=60` filter, budget 256), where full – random = 0.45 and `kv_norm` falls below or level with `random`. At that point, any real improvement should be obvious.

Before getting there we tried more aggressive budgets (160, 192) that overshot the useful point. With that much cache pressure the model solved nothing, correctness was 0 in every episode and PPO was left without a gradient again. This shows that the useful window is quite narrow. **The task has to remain solvable under eviction, but be hard enough that the eviction strategy still makes a difference.**

In the calibrated configuration the most interesting behaviour appears. The task becomes solvable, but only if the correct reasoning chain is preserved, and at the same time the space of eviction decisions is enormous. That creates a very marked exploration problem. Finding a consistent sequence of ~ 350 evictions (with ~ 220 options per step across $\times 28$ layers) is out of reach for online exploration from scratch.

Even artificially increasing the number of exposures per example and normalising the terminal reward with a *regret* baseline (to give PPO many more samples and remove the difficulty variance between examples), the *probes* show no learning. Training found correct episodes only by chance (~ 4 in 235 *rollouts*) and PPO never turned them into a policy.

The bottleneck appears to be in the reward signal, which is too sparse to guide effective exploration in this regime.

4.12 | The causal dense reward at scale

The exploration problem suggested that a dense reward (subsection 3.2) could help, since it assigns credit at every step without depending on correctly completing the whole episode. Recall how it is defined. At each step, the next-token distribution of the evicted cache is compared with that of a “shadow” cache that never evicts, and the divergence is penalised,

$$r_{\text{step}} = -w \text{clip}(\text{KL}(p_{\text{full}} \| p_{\text{evict}}), 0, c).$$

This was the first experiment where we evaluated it in a genuinely favourable scenario. The policy already had more informative features and training was done in the regime where there really was room to improve.

However, the result was parity again. Neither the MLP policy ($+0.021 \pm 0.014$, n.s.) nor the attention policy ($+0.016 \pm 0.018$, n.s.) shows a significant improvement over `kv_norm`.

The most interesting part appears when looking at what training is actually optimising. The per-step KL divergence does decrease steadily, that is, the dense reward is indeed being optimised. But that improvement never translates into higher *correctness* on the *probe*. In this regime, minimising the KL

divergence between the next-token distribution of the full cache and that of the evicted cache is not enough to improve performance. The policy learns to keep similar next-token distributions, even when the evictions have already broken the reasoning chain and end up leading to truncated or incorrect answers. In other words, the reward stops being aligned with the final objective.

4.13 | Per-layer credit

After all these results we went back over the implementation and there the explanation appeared for why every variant ended in parity. The problem was not in the representation, the architecture or the reward, but in how the environment assigned credit.

The environment exposes $n_envs = N \times 28$, so the policy takes **28 eviction decisions per step**, one per layer. However, the reward is a single scalar per episode assigned equally to all 28 layers. With the terminal reward this happens directly, but even with the dense reward the KL is computed over the model's final distribution, which depends jointly on the evictions performed in all 28 layers.

PPO therefore sees 28 different (state, action) pairs sharing a single return. The effect of changing the decision in one layer is mixed with the effect of the other 27, so the algorithm never receives information about which layer was really responsible for the outcome. In that scenario, the only thing it can learn over an action space of 220^{28} is a reasonable rule that works, on average, for every layer, which ends up looking very much like `kv_norm`. This explains why every variant ended at practically the same point.

Table 4.1: None of the axes we attacked touched per-layer credit.

What we tried	What it improved	Did it touch per-layer credit?
more informative features	representation	no
warm-start (cloning <code>kv_norm</code>)	initialisation	no
cross-token attention	architecture	no
causal dense reward	temporal credit (per step)	no (KL is joint)
50 repetitions + <i>regret</i>	samples + variance reduction	no

Looking back, having taken each variant to the end turned out to be important. Each experiment isolated a different hypothesis and allowed it to be ruled out with evidence. Only after going through all those possibilities did the cause appear, and it was in the environment itself.

4.14 | Per-layer reward and a second bottleneck

The solution to the credit assignment problem was fairly direct. Instead of using a single reward for all the layers, each one receives its own dense reward, computed from the marginal divergence of the *hidden state* that its eviction decision produces relative to a shadow cache that never evicts. Specifically,

$$\text{damage}_l = \text{relu}(\text{div}_{l+1} - \text{div}_l),$$

which isolates the contribution of layer l . The terminal reward is still shared by all the layers, but now a dense signal specific to each one is added.

We verified that the change was working as expected. Before, all 28 layers received exactly the same reward; after this modification, each one received a different value.

However, the results landed at parity again. Neither the MLP policy ($+0.021 \pm 0.035$) nor the attention policy ($+0.038 \pm 0.022$) showed a significant improvement over `kv_norm`. The only *probe* that reached $+0.19$ was an isolated peak that disappeared in the following evaluations.

This ends up showing that there were two overlapping problems. The first was per-layer credit assignment. But once that problem was solved another appeared. **The dense rewards we can compute online, whether the joint KL divergence or the per-layer hidden-state divergence, stop being aligned with the final objective under the truncation regime.**

Taken together, the results point to the same conclusion. Online PPO is limited by the training signal. First, the original reward could not assign credit correctly across layers. And even after fixing that point, the dense rewards available online still do not reflect faithfully enough which eviction decisions preserve

the correct answer, while the terminal reward remains too sparse to learn from directly. This also explains why recent works such as KVP and ForesightKV resort to offline supervision with future information instead of depending on online RL.

4.15 | The dataset as the limiting factor

Up to this point every experiment pointed in the same direction, but an important question remained. Were the results really due to our online training method, or could they have something to do with the dataset we were using?

Comparing our setup in detail with KVP's revealed a difference we had not isolated until then.

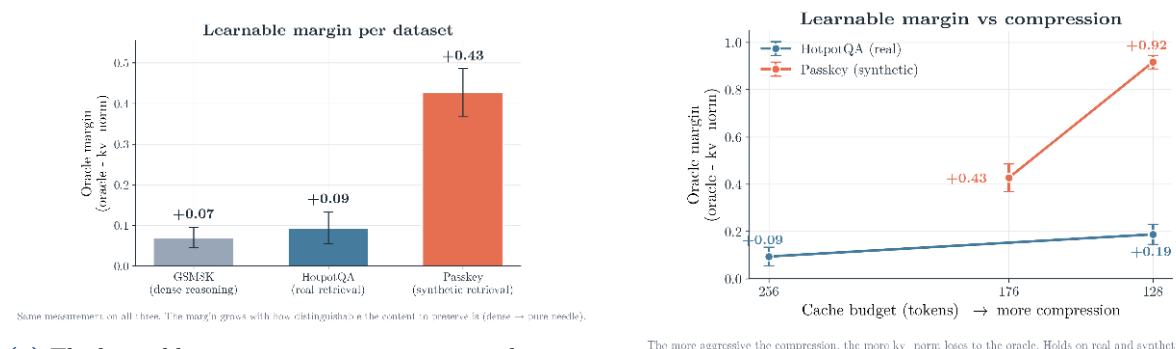
Table 4.2: Apple/KVP vs. our setup.

	Apple / KVP (RULER)	Ours (GSM8K)
task	long-context retrieval	short dense reasoning
useless filler	most of the prompt	almost none (dense)
decisive content	in the prompt (a “needle”)	generated while decoding
future utility predictable?	yes (by content)	no (depends on the reasoning)

GSM8K is, structurally, a very hard case for learning an eviction policy. The prompts are short and dense, there is practically no surplus information and the most important content appears during generation. As we saw in the previous section, that future utility cannot be inferred from the available features either, which is reflected in an agreement with the oracle of barely 3%. In RULER, by contrast, most of the context is filler and the *needle* can be told apart by its content before generation even starts (there future utility *is* predictable).

To check whether that difference was enough to explain the results, we built a reduced version of the RULER scenario called **passkey**. The dataset consists of a synthetic *needle-in-a-haystack* problem, with filler text, a secret code buried at a random depth and a task that forces the model to generate before retrieving it. On this dataset we ran exactly the same analysis with the future-attention oracle.

The behaviour changes very markedly (figure 4.7a). On **passkey**, the oracle beats **kv_norm** by +0.43 (7.3σ , 45 wins, 4 losses and 47 ties over 96 examples), whereas on GSM8K the difference was barely +0.07, using the same model, the same budget and exactly the same code. Moreover, **kv_norm** falls below **random** ($0.375 < 0.490$), which indicates that keeping the highest-norm tokens stops being a good strategy when what matters is preserving information that is distinguishable by content.



(a) The learnable margin as a spectrum across datasets. Difference between the future-attention oracle and **kv_norm**, with the same model, budget and implementation.

(b) The learnable margin grows with the aggressiveness of the compression. Oracle minus **kv_norm** as a function of the cache budget, on HotpotQA and **passkey**.

Figure 4.7: The learnable margin depends on the dataset and on the compression regime.

This result changes the interpretation of the whole series of experiments. The results on GSM8K may not be due to a problem with the algorithm, but to that dataset offering very little signal for learning an eviction policy. From this comparison, several characteristics that favour learning eviction policies

emerge: a high effective compression (ratio $\geq 3-4\times$), plenty of dispensable content, utility distinguishable by content rather than by reasoning, relatively short generations and an easily verifiable answer.

To check that this behaviour was not exclusive to the synthetic dataset, we repeated the same analysis on HotpotQA-distractor (multi-hop questions over Wikipedia paragraphs, with distractor paragraphs). There the oracle's margin is $+0.09$ ($z = 2.4$) at $4-5\times$ compression, and grows to $+0.19$ ($z = 4.4$) at $8-10\times$ compression. As cache pressure increases, `kv_norm` degrades until it approaches chance while the oracle keeps a clear advantage. On `passkey` the same effect is even more pronounced ($+0.43$ up to $+0.92$). In other words, the learnable margin *grows* with the aggressiveness of the compression (figure 4.7b), on both real and synthetic data, which confirms that the result depends on the structural properties of the dataset.

The three datasets can be arranged along a single spectrum (figure 4.7). At one end is GSM8K, dense reasoning where there is practically no margin. In the middle is HotpotQA, real retrieval where the margin is small but not zero and also grows with compression. And at the other end is `passkey`, pure synthetic retrieval where the margin is large.

Taken together, the learnable margin appears to depend on how distinguishable by content the information worth preserving is. It is almost nil when that information coincides with the very reasoning the model generates, and it grows when it is a specific datum hidden among filler text.

4.16 | Our method on a dataset with signal

Once a dataset where learned eviction has room to improve had been found, the question becomes whether our method (sequential online PPO) can exploit that signal or whether the offline training proposed by the literature is needed.

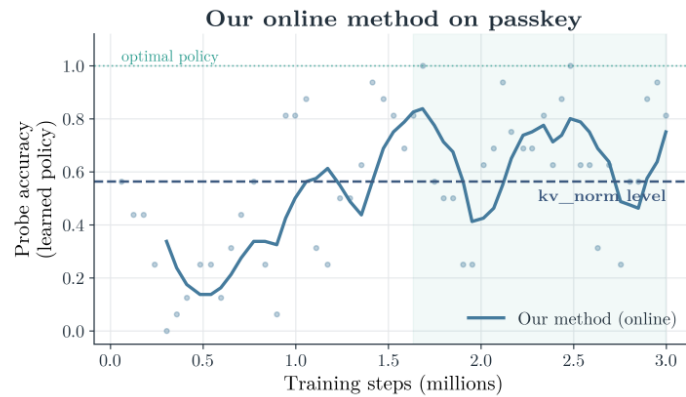
As a reference, we first reimplemented Apple's recipe on `passkey`: future-attention rankers trained offline over traces, with no online RL. In this scenario, it beats `kv_norm` by $+0.46$ (two seeds, 49 wins and 3 losses over 100 examples). This confirms that the task contains a learnable signal and serves as a reference for what can be captured on this dataset.

We then evaluated our online formulation. The first attempt, using only the terminal correctness reward, showed interesting behaviour. The policy reached, at moments, a perfect eviction policy (getting 100% of the *probe* right, something that had never happened on GSM8K), but that performance was not sustained. Training oscillated continuously, alternating between peaks above `kv_norm` and drops to values close to zero. Since the signal demonstrably exists, this pointed to a training *stability* problem.

The next step was to train with the causal dense reward proposed in subsection 3.2 and to try different configurations of learning rate, exploration and number of optimisation epochs per *rollout*. Every run uses the same *setup*: the `passkey` dataset with a cache budget of 300 tokens, forced generation of up to 300 tokens so that the *needle* is under real eviction pressure, the causal dense reward with weight 0.05 and clip 5.0, and 3M training steps. The only thing that changes between configurations is PPO's hyperparameters.

With one of those configurations (learning rate 10^{-4} and 10 optimisation epochs per *rollout*) a different behaviour starts to appear (figure 4.8). During the first half of training the policy oscillates between 0 and `kv_norm`, with a mean of 0.36. In the second half, the centre of that oscillation shifts upwards and the mean rises to 0.67, above `kv_norm` (0.56). Two thirds of the evaluations in that second half land above the heuristic and the best *probe* reaches a perfect policy (100% correct).

In other words, as training advances the policy starts spending more time above `kv_norm`, something we had observed neither on GSM8K nor on this same task using only the terminal reward.



Points: raw probes. Line: trend (moving average). The band marks the second half, where the centre rises above `kv_norm`.

Figure 4.8: Sequential PPO with the causal dense reward on **passkey**. Accuracy of the learned policy on the *probe* over the course of training; points are individual evaluations and the line is the trend. In the second half the centre of the oscillation shifts above `kv_norm` and the policy reaches the optimum (1.0), though it still oscillates.

It is important to make clear that this is not yet a stable convergence. Even in the second half of training the policy keeps oscillating and evaluations appear that fall back to `kv_norm` or even below. The improvement appears as a shift in average behaviour and not as a convergence. For the first time the policy manages to stay above `kv_norm` for several consecutive evaluations, even reaching a perfect eviction policy.

This result also has two important limitations. The first is sensitivity to hyperparameters. Of the three configurations we tried, only this one shows the upward shift. The second is that, for now, the evidence rests on a single seed. More iterations and additional configurations would be needed to validate the stability of the phenomenon.

More runs are needed before asserting a firm conclusion. But the available evidence suggests that the online reformulation proposed in this work, which on GSM8K remained systematically at parity with `kv_norm`, can beat that heuristic when the dataset offers a learnable signal.

5 | Conclusion

This work started from a concrete question: whether learned KV-cache eviction, which the recent literature trains with bespoke implementations such as Apple’s (KVP, subsection 2.1), could be reformulated so as to be expressible as a standard Gymnasium environment and trainable with Stable-Baselines3’s masked PPO. The central idea was to turn eviction, which in KVP is a single-pass ranking of the whole cache, into a *sequence* of discrete decisions, and moreover a dynamic one, as the model generates tokens. That reformulation did turn out to be trainable, which is already a step beyond the state of the art.

The central empirical result is that the answer to that question depends on the dataset, not on the method. On GSM8K, no axis we attacked (representation, architecture, initialisation, credit assignment, reward density) took the policy beyond parity with `kv_norm`. But measured with a future-attention oracle, which no online policy can reach, the margin separating `kv_norm` from optimal eviction on GSM8K is barely +0.07; on retrieval tasks, real (+0.09 to +0.19 depending on the compression) or synthetic (+0.43), that margin is several times larger. GSM8K, precisely the dataset chosen to test eviction *during* generation and not only over a fixed context, turned out to be the worst possible case for this problem. Once a dataset with signal was located, our own formulation, online, with the causal dense reward and a configuration that stabilises training, shows there a clear shift above `kv_norm` (though not yet stable, and resting for now on a single seed).

This also lets us revisit the design decision discussed in subsection 2.2: keeping the language model frozen and training an external, lightweight policy, instead of training the whole model as NGC does. A policy that only sees *K* and *V* has no access to everything the model “knows” internally about a token’s future utility. An approach like NGC, which retrains the model while deciding what to forget, could exploit that internal information. The contrast between the two approaches is clearer after this work than

before: it is not only a question of computational cost, but of what information is available depending on where the policy sits. With more time and compute the two worlds could be joined into a method unifying the most advanced techniques in the state of the art.

5.1 | Contributions

This work makes two main contributions. First, a sequential reformulation of learned eviction that makes it expressible as a Gymnasium environment and trainable with standard masked PPO, together with the online eviction environment over a frozen model that implements it. And second, the central finding: the conditions under which learned eviction can beat a heuristic, together with preliminary evidence that our own online reformulation can exploit them.

5.2 | Limitations and future work

The work has three limitations that bound its conclusions. The first is one of scope: every experiment was run on a single model (Qwen2.5-1.5B-Instruct) and moderate cache budgets, and absolute accuracy can depend on the attention backend used in each run, so only the paired contrasts within a single run are comparable with one another. The second concerns the causal dense reward: it rewards the policy for resembling, step by step, a model that never evicts, but that is not the same as directly rewarding the correctness of the final answer; an eviction can shift the token distribution only slightly and still change the answer, or shift it a great deal without affecting it. That decoupling between what the reward measures and what ultimately matters is, in part, what keeps making training difficult even when there is signal to exploit. The third is that the positive result of our online method is still preliminary: it rests on a single hyperparameter configuration and, for now, on a single seed, and it needs confirming before being treated as a firm conclusion.

Several directions follow from this. The most immediate is to confirm the online result with more seeds and longer training, and to incorporate PPO stabilisation techniques to turn the observed shift into sustained convergence. In parallel, it would be valuable to take the environment to real long-context datasets, where the learnable margin measured by the oracle is larger, and to explore per-layer credit assignment based on correctness and not only on distributional divergence. Finally, the contrast with approaches that train the model and the policy jointly, such as NGC, remains a natural comparison for understanding how much of the difficulty of this work comes specifically from learning to forget over a model that cannot adapt to that constraint.

6 | References

- [1] Arash Ahmadian, Chris Cremer, Matthias Gallé, Marzieh Fadaee, Julia Kreutzer, Olivier Pietquin, Ahmet Üstün, and Sara Hooker. Back to basics: Revisiting reinforce style optimization for learning from human feedback in llms. *arXiv preprint arXiv:2402.14740*, 2024. <https://arxiv.org/abs/2402.14740>.
- [2] Greg Brockman, Vicki Cheung, Ludwig Pettersson, et al. Openai gym. *arXiv preprint arXiv:1606.01540*, 2016. <https://arxiv.org/abs/1606.01540>.
- [3] Karl Cobbe, Vineet Kosaraju, Mohammad Bavarian, et al. Training verifiers to solve math word problems. *arXiv preprint arXiv:2110.14168*, 2021. <https://arxiv.org/abs/2110.14168>.
- [4] Wouter Kool, Herke van Hoof, and Max Welling. Stochastic beams and where to find them: The gumbel-top-k trick for sampling sequences without replacement. In *Proceedings of the 36th International Conference on Machine Learning (ICML)*, 2019. <https://arxiv.org/abs/1903.06059>.
- [5] Maxime Labonne. Qwen3.5: Nobody agrees on attention anymore. Hugging Face Blog, 2026. <https://huggingface.co/blog/mlabonne/qwen35>.
- [6] Michael Y. Li, Jubayer Ibn Hamid, Emily B. Fox, and Noah D. Goodman. Neural garbage collection: Learning to forget while learning to reason. *arXiv preprint arXiv:2604.18002*, 2026. <https://arxiv.org/abs/2604.18002>.

- [7] Yuhong Li, Yingbing Huang, Bowen Yang, et al. Snapkv: Llm knows what you are looking for before generation. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2024. <https://arxiv.org/abs/2404.14469>.
- [8] Luca Moschella, Laura Manduchi, and Ozan Sener. Learning to evict from key-value cache. In *Proceedings of the 43rd International Conference on Machine Learning (ICML)*, 2026. arXiv:2602.10238. <https://arxiv.org/abs/2602.10238>.
- [9] Robin L. Plackett. The analysis of permutations. *Journal of the Royal Statistical Society: Series C (Applied Statistics)*, 24(2):193–202, 1975. <https://doi.org/10.2307/2346567>.
- [10] Qwen Team. Qwen2.5 technical report. *arXiv preprint arXiv:2412.15115*, 2025. <https://arxiv.org/abs/2412.15115>.
- [11] Antonin Raffin, Ashley Hill, Adam Gleave, Anssi Kanervisto, Maximilian Ernestus, and Noah Dormann. Stable-baselines3: Reliable reinforcement learning implementations. *Journal of Machine Learning Research*, 22(268):1–8, 2021. <https://www.jmlr.org/papers/v22/20-1364.html>.
- [12] John Schulman, Filip Wolski, Prafulla Dhariwal, Alec Radford, and Oleg Klimov. Proximal policy optimization algorithms. *arXiv preprint arXiv:1707.06347*, 2017. <https://arxiv.org/abs/1707.06347>.
- [13] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Łukasz Kaiser, and Illia Polosukhin. Attention is all you need. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2017. <https://arxiv.org/abs/1706.03762>.
- [14] Guangxuan Xiao, Yuandong Tian, Beidi Chen, Song Han, and Mike Lewis. Efficient streaming language models with attention sinks. In *International Conference on Learning Representations (ICLR)*, 2024. <https://arxiv.org/abs/2309.17453>.
- [15] Zhenyu Zhang, Ying Sheng, Tianyi Zhou, et al. H2o: Heavy-hitter oracle for efficient generative inference of large language models. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2023. <https://arxiv.org/abs/2306.14048>.